

Consignas generales de los trabajos prácticos

Criterios de evaluación

- Implementación de lo solicitado
- Cumplimiento de las fechas de entrega
- Organización y prolijidad del proyecto
- Utilización de las herramientas y buenas prácticas
- Lenguaje técnico y buena redacción

Reglamento de entregas de los avances del trabajo práctico de la materia

- Las fechas de entrega están en el cronograma general de la materia con fecha límite.
- Los avances e informes se entregan semanalmente o en la fecha dispuesta y notificada por el docente.
- Salvo excepciones justificadas, se descuentan puntos por entregas tardías.
- Las entregas se promedian para las notas intermedias o finales, según corresponda.
- Las notas son INDIVIDUALES

Índice

Criterios de evaluación 1
Reglamento de entregas de los avances del trabajo práctico de la materia 1
Índice 2
Desarrollo 2
CONSIGNAS 3
Etapas: 3
1. TP 1: Desarrollo de la propuesta por escrito en un proyecto 3
2. TP 2: Back-end para servir el sitio web 4
3. TP 3: Creación del CRUD para administrar Front-End 5
4. TP 4: Implementación de autenticación, accesos y seguridad 6
5. OPCIONAL: “Despliegue a producción” 7
Herramientas y bibliografía 7

Desarrollo

En total, los trabajos prácticos son 4, que corresponden a las 4 etapas de un mismo proyecto **Back-End**.

Los trabajos corresponden a un desarrollo **Back-End completo**, integrando el **Front-End** de AW1, o un nuevo Front-End que ustedes desarrollen con las mismas características del trabajo de AW1.

IMPORTANTE: Si bien se utilizan herramientas de inteligencia artificial, cada integrante del equipo de desarrollo debe conocer cabalmente su propio trabajo.

CONSIGNAS

Etapas:

(ver cronograma para fechas límite)

1. TP 1: Desarrollo de la propuesta por escrito en un proyecto

El desarrollo del proyecto es una propuesta de trabajo que debe incluir obligatoriamente:

1.1. Equipo y roles (salvo que se trabaje solo o sola) que pueden ir rotando.

1.2. Lo que se va a desarrollar y su justificación (¿por qué es necesario y qué problema resuelve o qué propuesta de valor brinda?).

1.3. Cronograma de avances y presentaciones. Un documento que dé cuenta de las fechas (deadline) para las entregas. Utilización de una herramienta de organización de tareas.

1.4. Uso de una herramienta de seguimiento y de control de versiones. *(El proyecto seguramente va a tener modificaciones; lo importante es la organización del desarrollo y las entregas pautadas).*

1.5. **Entregas**:

1.5.1. *Link al repositorio (las fechas de commits deben estar dentro de las fechas de entrega).*

El repositorio debe contener:

1.5.1.1. PDF del Proyecto (TP1) .

1.5.2. Link a la aplicación de seguimiento de proyectos.

2. TP 2: Back-end para servir el sitio web

En esta etapa se crea un servidor web básico para publicar la web que han trabajado en AW1 o uno con las mismas características.
La web debe estar totalmente funcional y tomar los datos desde una API externa.

2.1. Se trabajará con NodeJS y ExpressJS.

2.2. La vinculación a los datos se hará desde una plataforma como MockAPI o similar.

2.3. El proyecto debe estar organizado (archivos y directorios) de manera coherente y de fácil mantención y escalado, apoyándose en la modularización y división de responsabilidades.

2.4. Informe de desarrollo: Compilación de informes entregados más el actual.

2.5. **Entregas (al formulario Google designado)**:

2.5.1. *Link al repositorio (las fechas de commits deben estar dentro de las fechas de entrega).*

El repositorio debe contener:

2.5.1.1. *Código.*

2.5.1.2. *PDF del Proyecto (TP1) y los sucesivos informes de avances (TODO EN EL MISMO PDF).*

2.5.1.3. *Agregar al documento la descripción de los 2 endpoints (rutas) de la API y su método o verbo con una mínima referencia:*

Ejemplo:

[GET] (base-url)/productos

Endpoint para obtener todos los productos.

2.5.2. Link a la aplicación de seguimiento de proyectos.

3. TP 3: Creación del CRUD para administrar Front-End

Tener en cuenta que todas las API estarán desplegadas bajo el mismo dominio y puerto; por lo tanto, deberán encargarse de realizar bien el enrutado.

3.1. Organizar el proyecto utilizando un patrón mínimo de MVC[1] visto en clase. La organización debe ser por módulo[2], además de utilizar un enrutamiento con Router. Pueden mejorar el patrón utilizando más capas[3].

3.2. Creación de API:

3.2.1. CRUD con 5 endpoints (2 LECTURAS, ALTA, BAJA y MODIFICACIONES) de productos/servicios.

3.2.2. Exponer los endpoints de lectura. Habilite CORS para ambos si es necesario.

3.3. Creación de la interfaz Front-End para la administración.

3.4. Utilización de una BASE DE DATOS que almacenará los ítems.

3.4.1. Puede ser PostgreSQL o MongoDB.

3.5. Documentar la API con Swagger (ver sitio web) o Scalar (ver sitio web).

3.6. **Entregas (al formulario Google designado)**:

3.6.1. Link al repositorio (las fechas de commits deben estar dentro de las fechas de entrega).

El repositorio debe contener:

[1] Modelo Vista Controlador.

[2] En clase vimos la organización por módulo, agrupando las capas dentro del mismo. Por ejemplo, en un módulo de productos:
- productos/modelo.productos.mjs
- productos/controlador.productos.mjs

[3] Por ejemplo, una capa de **Presentacion** de los datos, es decir, su serializacion para enviar en formato JSON al cliente. También puede haber una capa de **Servicios**, donde la lógica de negocios se mueve del **Controlador** a esta capa, etc.

3.6.1.1. Código.

3.6.1.2. PDF del Proyecto (TP1 , TP2 y TP3) y los sucesivos informes de avances (TODO EN EL MISMO PDF).

3.6.1.3. Incluir archivo SQL o Mongo para replicar la BD.

3.6.2. Link a la aplicación de seguimiento de proyectos.

4. TP 4: Implementación de autenticación, accesos y seguridad

4.1. Crear persistencia de datos en BASE DE DATOS para usuarios registrados. *(Crear tablas o colecciones según corresponda al tipo de Base de Datos SQL o NoSQL).*

4.2. Utilizar ***hashing*** de claves para mejorar la seguridad de las credenciales.

4.3. Implementación de autenticación y control de acceso a través del uso de ***JWT*** y cookies.

4.4. Habilitar ***CORS*** si es necesario (debe estar justificado).

4.5. Uso de ***variables de entorno*** para no exponer datos sensibles como claves de bases de datos, tokens privados, claves secretas, etc.

4.6. Ampliar la documentación de la API con Swagger o alternativa.

4.7. **Entregas**:

4.7.1. Link al repositorio (las fechas de commits deben estar dentro de las fechas de entrega).

El repositorio debe contener:

4.7.1.1. Código.

4.7.1.2. PDF del Proyecto y los sucesivos informes de avances (TODO EN EL MISMO PDF).

4.7.1.3. Incluir archivo SQL o Mongo para replicar la BD.

4.7.2. Link a la aplicación de seguimiento de proyectos.

5. OPCIONAL: *"Despliegue a producción"*

Herramientas y bibliografía

Para lo teórico, se deben apoyar en los textos de AW1 y AW2.

Para las herramientas:

- Node, ExpressJS, PostgreSQL, MongoDB. Opcionales: Docker (local), Supabase (nube), otras.
- Control de versiones: Git y GitHub.
- Swagger https://swagger.io/
- Scalar https://scalar.com/
- Seguimiento y organización de proyectos y creación de informes: Trello o github projects,
- Herramienta de mock para API TP2: https://mockapi.io/ (otras)